

System 12: In-Memory File System

Zero-Prerequisite Intuition: The "Nested Russian Doll" Metaphor What is a file system? Imagine a massive physical library. You have folders inside boxes, and boxes inside cabinets. Some items in the cabinet are actual **documents** (files with text written on paper), while other items are **folders** that hold more documents or sub-folders.

When your computer boots, your operating system organizes data on a hard drive using a hierarchical tree structure: `/` (the root cabinet), `/var` (a drawer), and `/var/log/app.log` (a physical text page).

In top-tier software engineering interviews (Google, Dropbox, Amazon), candidates are asked: *"Design and implement an in-memory file system from scratch that supports `mkdir`, `ls`, `create_file`, `read_file`, `write_file`, and nested path resolution."*

This is the ultimate test of the **Composite Design Pattern** and **Trie/Tree Traversal**.

1. Requirements & System Boundaries

Functional Requirements

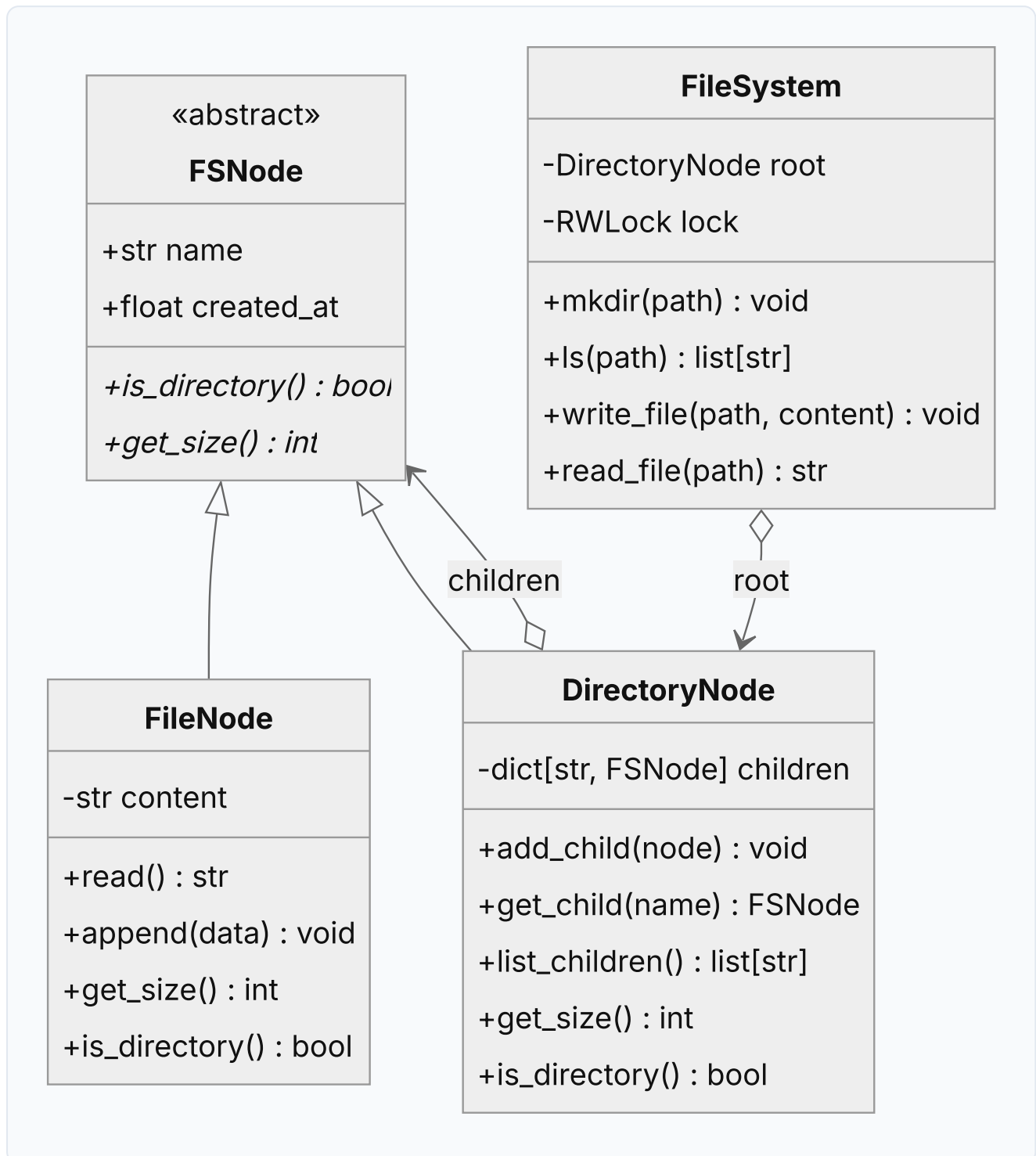
1. `mkdir(path)` : Creates a new directory at the specified absolute path. Supports recursive creation (e.g. creating `/a/b/c` automatically creates missing parents).
2. `ls(path)` : Lists the contents of a directory in alphabetical order. If the path points to a file, returns just the file name.
3. `write_file(path, content)` : Writes or appends text content to a file. If the file does not exist, creates it.
4. `read_file(path)` : Reads and returns the complete string content of the file.
5. `delete(path)` : Recursively deletes a directory or file.

Non-Functional Requirements

- **Thread Safety:** Multiple threads can read concurrently; write mutations (`write_file` , `mkdir` , `delete`) must be atomic without corrupting the tree.
 - **$O(L)$ Time Complexity:** Operations scale with path depth L (number of `/` segments), not total files in the system.
-

2. Design Pattern Blueprint: The Composite Pattern

A directory contains files and other directories. Instead of treating files and directories as totally separate, the **Composite Pattern** treats them uniformly as `Node` objects.



3. Complete Python Implementation



4. Verification & Common Interview Follow-Ups



Classic Senior Interview Follow-Ups

1. "How do you support Hard Links and Symbolic Links (Symlinks)?"

- *Answer:* Introduce a `LinkNode(FSNode)` that stores an absolute or relative target path string. When resolving paths, `FileSystem` follows the target path recursively (with a `max_depth = 16` counter to prevent infinite cyclic link loops).

2. "How do you implement permissions (`chmod` , `chown`)?"

- *Answer:* Add an integer bitmask `permissions = 0o755` (Read/Write/Execute for Owner, Group, Other) to `FSNode` . Check `user_id` and permission bits inside every `read_file` or `write_file` method.